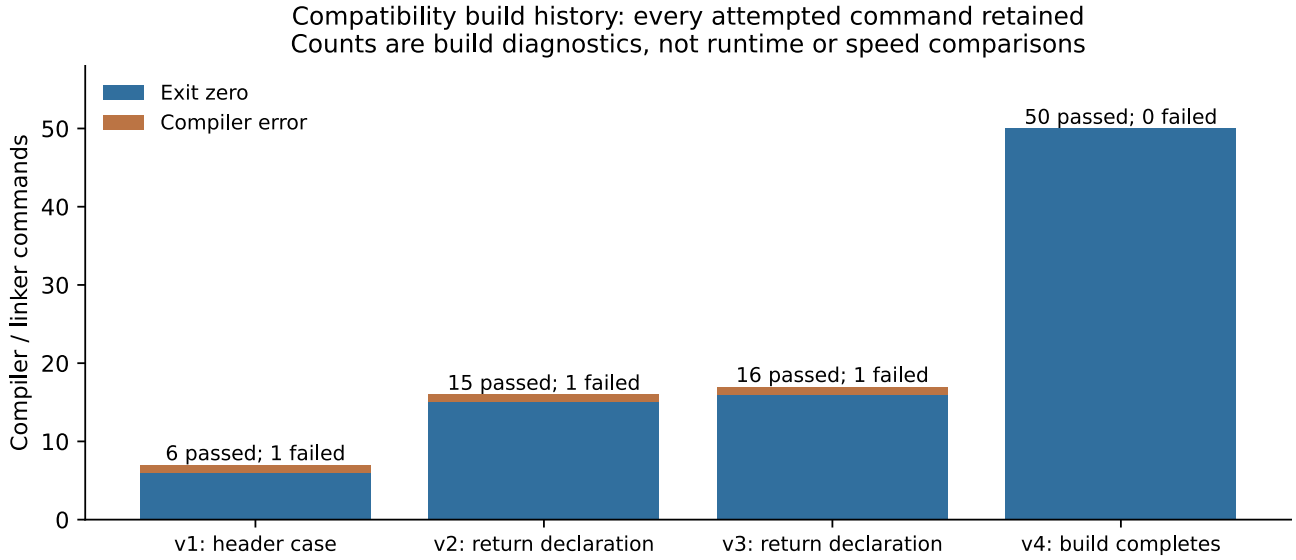
ECPP compatibility controls make one failure path concrete

24 October 2026 UTC. Engineering controls; zero named NIST/ANSI orders executed.

The recovered ECPP 3.4.1 source builds as a static 32-bit Linux executable. All 345 integer comparisons pass; four ordinary small classifications are correct; and an elliptic-curve-only proof succeeds for prime 101. With an intentionally empty discriminant catalog and both N±1 routes disabled, the same executable calls 101 composite and exits zero. A missing catalog instead produces a segmentation fault.

This demonstrates an induced software failure path. It does not classify any of the eleven omitted curves or establish that the original generator used this program.



All compiler/linker commands in four retained build attempts. Command counts are diagnostics, not speed comparisons. Raw receipts: attempts/build-v1 through build-v4. Editable graph: render.py.

The build preserves 32-bit int, long, pointer and BigNumDigit. It selects the portable C kernel and serial route, adds declarations matching existing definitions and corrects an include-order collision on the case-insensitive host. Source/data bodies remain unchanged. Zig 0.16.0 targets x86-linux-musl; local Linux/386 containers use read-only inputs/root, no network, dropped capabilities, one CPU and 128 MiB. This does not reproduce a historical compiler, processor, libc or RNG.

Completed control	Observation
69 integer vectors	345 exact add/subtract/multiply/divide/remainder comparisons, through 521 bits
Ordinary small inputs	101 / 65537 prime; 15 / 341 composite
CM-only 101	D=4 path succeeds; a=21, b=0, P=(20,80) over F101
Independent affine check	Exact point count 104; point order 26
Empty catalog / 101	Two exhaustion messages, one retry, composite label, exit zero
Missing catalog / 101	Segmentation fault, exit 139, no final label

The separate summary replays integer outputs, small classifications and the control curve's affine arithmetic and point count. The CM certificate's pending legacy-checker call must still finish. Timings remain in raw receipts; CPU and peak RSS were not measured. Resource caps are configuration, not measured consumption.

Failed attempts and a certificate discrepancy stay visible

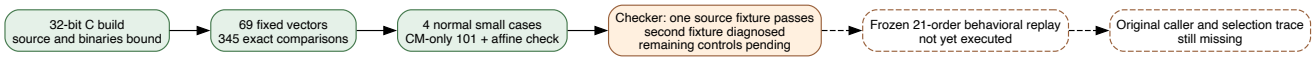
The first three builds fail on header resolution or return declarations. The first build launcher also returned zero after recording its compiler error; later launchers return nonzero on failure. Two arithmetic-admission attempts stop before execution because of Docker platform/image addressing. The third passes. A small-control parser misses the percent-prefixed composite message; its native output is reused by the corrected parser, with no repeat of the first three processes.

The independently compiled legacy checker accepts one bundled certificate and rejects the first two steps of the other. The initial positive expectation fails and remains preserved. Integer diagnosis shows the second example's listed factors leave a cofactor of 1 where the current theorem-3 reader expects a prime cofactor. A separately named variant leaves 1583 and 113; both intended modular conditions are checked independently. The original sample, checker and algorithm remain unchanged. Whether that example was stale or intentionally negative is unknown.

The remaining checker continuation did not execute a fixture

The variant and remaining checker controls wait in Docker image metadata before a started record is written. Bounded Docker health queries time out. After revalidating the exact PIDs, the metadata child is terminated under a watchdog and the parent returns an infrastructure error. No certificate result is inferred from it.

System-volume exhaustion is observed, followed later by recovered space; its causal relationship to Docker remains unknown. Completed temporary source/render objects are moved byte-for-byte to the data volume with original paths preserved as symlinks. The shared Docker service is not restarted. An independent local runtime can supply the next route after its own declared controls.



Solid edges mark completed compatibility steps. Dashed edges mark work still required. This is a process diagram, not a historical execution trace.

The historical-selection question remains open. Complete checker/launcher admission and freeze the full 21-order behavior comparison before executing named inputs. Original catalog/tactics, exact curve identities, fresh state, process limits and immutable results remain required. A split would still need wider prediction, independent validation and original-caller provenance. All-success would exclude only the tested configuration.

No shared mathematical weakness, ECDLP cost advantage, scientific-status change or new frontier edge is established. Existing frontier and selection graphs were checked and remain unchanged.

Bindings and reproduction

Original archive SHA-256: 0387e7d180e570d0cf1ff957ed281a46711fb20d50ae0b0669a01a3464d9160f. Source acquisition and its historical-authentication limits are in ../nist_seed_ecpp_source_20261024/report.md. Build scripts, protocols, corrections, raw outputs, binary hashes and local retention paths are in report.md and summary.json. Recompute the completed summary using summarize.py, then render.py, Graphviz and Typst. Both PDF pages were visually inspected.

This work uses the BigNum package developed jointly by INRIA and Digital PRL. Copyright notices remain in the local source copies; the external package is not redistributed here. The interactive model identity was not exposed in these command receipts and is not inferred.